

HelixPlay

HelixPlay

Tagline: Turn any GPU machine into your own cloud-gaming appliance.

Summary: HelixPlay is a self-hostable cloud-gaming platform that turns any GPU-equipped machine into a remote streaming host, delivering console-class gameplay to desktop, mobile, TV, and browser clients. It is built as a Go-centric monorepo of 46 submodules with a triple-stack client and is white-labellable for partners.

Short description (~40 words): HelixPlay is a self-hostable, open, white-labellable cloud-gaming platform. It turns any GPU-equipped machine into a remote streaming host and delivers console-class gameplay to desktop, mobile, TV, and browser clients over WebRTC/QUIC, with a Go core and a Wails/Flutter/Angular client stack.

Long description (150-250 words): HelixPlay is a cloud-gaming platform built as a Go-centric monorepo composed of 46 Git submodules. It transforms any gaming PC into a streaming host and delivers PS4-Pro-class user experience to desktop, mobile, TV, and browser clients — self-hostable, open, and white-labellable for partners.

Its defining design choice is triple-stack client convergence: a Wails desktop app, a Flutter mobile/TV app, and an Angular web client all share a single Go core (compiled to WASM for the browser). The media path runs Capture → Encode → Packetize → Transmit → Decode → Render, using platform-native capture (DXGI / ScreenCaptureKit / PipeWire) and hardware encoders (NVENC / QSV / AMF / VideoToolbox), transported over WebRTC (Pion v4), QUIC (quic-go), and custom UDP datagrams. The backend core handles sessions, tenants, catalog, and auth, with a host agent doing capture/encode/transport and mDNS/rendezvous for discovery.

HelixPlay is architected for white-label SaaS — per-tenant theming, catalog filtering, OAuth2, and billing — and is container-native, running every service, database, build, test, and scan

inside containers. Like the rest of the Helix family, it is governed by an anti-bluff constitution: green tests are meant to guarantee real, end-user-usable behavior. Its README states aggressive latency targets (≤ 30 ms LAN / ≤ 50 ms WAN p999) and a large multi-dimensional test matrix; these are stated design targets rather than independently confirmed benchmarks.

Why we built it: Commercial cloud gaming is closed, centralized, and rented. HelixPlay was built so anyone with a GPU machine can run their own streaming host — open, self-hosted, and white-labellable — instead of depending on a third-party service.

Why it's a game-changer: It combines self-hosting, one shared Go core across three client stacks, and white-label multi-tenancy — letting a partner stand up a branded cloud-gaming service on their own hardware rather than reselling someone else's cloud.

What's innovative: - **Triple-stack client convergence** — Wails + Flutter + Angular over one Go core (WASM in the browser). - **Self-hostable, white-label SaaS** — per-tenant theming, catalog, OAuth2, billing. - **Modern low-latency transport** — WebRTC (Pion) + QUIC + custom UDP with hardware-encoder selection per platform. - **46-submodule decoupled architecture** with container-native everything.

Biggest technical challenges + how solved: - *Low-latency streaming across heterogeneous hardware* — solved with a platform-aware capture/encode path (DXGI/SCK/PipeWire + NVENC/QSV/AMF/VideoToolbox) over WebRTC/QUIC/UDP. - *One product across desktop, mobile, TV, and web* — solved with triple-stack clients sharing a single Go core (compiled to WASM for the browser). - *Multi-tenant white-label operation* — solved with per-tenant theming, catalog filtering, OAuth2, and billing in the core backend.

Tech stack: - **Go (1.26.2 root / 1.25+ submodules)** — shared core backend and host agent. - **Wails v2** — desktop client (Go + embedded webview). - **Flutter 3.29+** — mobile/TV client (FFI to the Go core). - **Angular 17+** — web client (Go compiled to WASM). - **WebRTC / Pion v4, QUIC / quic-go, custom UDP** — real-time media transport. - **Hardware encoders** (NVENC / QSV / AMF / VideoToolbox) and platform capture (DXGI / ScreenCaptureKit / PipeWire) — the encode/capture path. - **Containers (Docker/Podman)** — every service, DB, build, test, and scan runs containerized. - **mDNS / rendezvous** — host discovery.

Public links: - GitHub: <https://github.com/HelixDevelopment/HelixPlay> (public) - License: no LICENSE detected via the GitHub API — UNVERIFIED / not declared.

Suggested diagrams/illustrations (OpenDesign): 1. Media pipeline: Capture → Encode → Packetize → Transmit → Decode → Render, annotated with the per-stage technologies. 2. Triple-stack convergence: Wails / Flutter / Angular clients all pointing at one Go core. 3. Self-hosted topology: a GPU host + host agent streaming over WebRTC/QUIC to phone, TV, laptop, and browser. 4. White-label layer: one platform, multiple branded tenant skins.

Site relevance: both (vasic.digital as a flagship platform product; milosvasic.ru as a high-impact systems/media-engineering highlight).

Priority tier: Helix-primary.

Source provenance: - gh api repos/HelixDevelopment/HelixPlay (metadata: public, Go, no license, created 2026-04-27, pushed 2026-05-03, size ~59 MB). - gh api repos/HelixDevelopment/HelixPlay/readme (overview, architecture, media pipeline, tech-stack table, 46 submodules). - /Volumes/T7/Projects/vasic/_analysis/github-helix-others.{md,json} (description, “46 submodules”, cloud-gaming positioning). - UNVERIFIED: license not declared. Latency targets (≤ 30 ms LAN / ≤ 50 ms WAN p999), “PS4-Pro-class”, and the test-matrix cell count are the project’s own README claims/targets, not independently benchmarked — presented above as stated targets.